3-Party Asynchronous Communication with Perfect Secrecy based on Periodic Pad Redistribution

Team members:

Willem Lessig: Protocol logic and implementation, report writing
Gaurav Rane: Multiparty channel implementation, proofs, report writing

Main Claims:

(a) Wasted pads
Maximum over all usage schedules: No matter how often each party sends, the protocol wastes at most d + m − 1 pads (as a function of m, n, d, L and the redistribution interval k).
Scenarios S.x (m = 3): In experiments with a single pad sequence per run and random senders, the average number of wasted pads is well below ((m−1)/m)n: typically on the order of a few percent of n for S.1 (one sender), and lower for S.2 and S.3 (e.g. S.3 often around 1–2% of n). So the protocol does better than the naive "split into m equal parts" solution, which can waste up to ((m−1)/m)n pads.
(b) Runtime to send one message
A party's work to send one L-pad message is O(L):
O(L) to choose the next L pad indices from its local list and check that they are not in use (e.g. via a global or agreed "used" structure),
O(L) to mark those pads as used and append one message to the in-flight set.
This is independent of n, m, and d; it is linear in L only. So for L = 1 (one pad per message), sending a single message is O(1).

M chosen: We chose m = 3 as our main design. We also implemented and reported results for m = 4 (extra credit ec1) and for larger m (extra credit ec2).

Explanation
Our protocol aims to allow *m=3* parties to send messages asynchronously using a shared sequence of *n* one-time pads, while ensuring perfect secrecy by ensuring no pad is ever reused and minimizing pad waste.
This is achieved by periodically redistributing the pad sequence among parties. Between redistributions, each party uses only its local list of pads to use, without coordination. After every *k* messages, all parties synchronize and redistribute the unused pads among themselves. This ensures that pads are not wasted due to uneven sending of messages. The protocol allows for up to *d* undelivered messages to model the asynchronous nature of a network. We did not simplify the problem beyond the assignment: we assume each message is delivered simultaneously to all m−1 recipients, as allowed.

How it works
1. At the start. *n* pads are split equally among m parties.

2. Each party sends messages using the next *L* pads from its list, marking them as “in flight”.
3. When a party cannot send, or after *k* messages, the parties synchronize, all in-flight messages are delivered and unused pads are redistributed equally.
4. This process repeats until no party can send messages securely.

Pseudocode

```
Parameters:
  m: number of parties (3)
  n: total pads
  d: max undelivered messages
  L: pads per message (1)
  REDISTRIBUTE_EVERY: number of messages between redistributions

State:
  For each party i:
    indices[i]: list of pad indices assigned to party i
    offset[i]: next index in indices[i] to use

  pad_owner: array of length n, tracks which party owns each pad (None if unused)
  in_flight: list of undelivered messages

Protocol:

1. Initialization:
   - Split n pads equally among m parties.
   - Each party i gets indices[i] = its segment of pads, offset[i] = 0.

2. Sending a message (by party i):
   - If offset[i] + L > len(indices[i]), cannot send (list exhausted).
   - pads = indices[i][offset[i] : offset[i] + L]
   - For each pad in pads:
       - If pad_owner[pad] != None, cannot send (pad conflict).
   - For each pad in pads:
       - pad_owner[pad] = i
   - offset[i] += L
   - Add message to in_flight.

3. Delivery:
   - When in_flight exceeds d, deliver oldest message (remove from in_flight).

4. Redistribution (every REDISTRIBUTE_EVERY messages or when no party can send):
   - Deliver all in_flight messages.
   - Compute set of unused pads (not in pad_owner).
   - Split unused pads equally among m parties.
   - For each party i:
       - indices[i] = new segment
       - offset[i] = 0

5. Termination:
   - Protocol ends when no party can send securely.
```

Proof
At any time, up to *d* pads may be "in flight" from undelivered messages, so at least d pads may be wasted.
After redistribution, unused pads are split as evenly as possible. If the number of unused pads is not divisible by *m*, up to *m−1* pads may be left unassigned in the final redistribution.
Therefore, the maximum number of wasted pads is ***d+m−1***.

Readme (Protocol)
From the project root:

```
python LessigRane-3-protocol.py
```

This runs a **dummy demo** that shows the protocol step-by-step:

- **Setup:** n=600, m=3, d=5, L=1, redistribute every 50 messages (see REDISTRIBUTE_EVERY in protocol). Run continues until no party can send (no artificial round limit).
- **Verbose output:** The **first 10 rounds** are printed in full: each round shows which party sends (and which pads), when in_flight exceeds d (and one message is delivered), and when a redistribution happens (sync + new list lengths). After that, one line says that later rounds are omitted and the run continues to completion.
- **End:** Prints “Stop: no party can send …” and a **Done** line with total rounds, wasted pads (as count and % of n), number of redistributions, and chat sim deliveries. This demonstrates that waste is a small fraction of n (e.g. a few percent) when the protocol runs to completion.

So the demo both illustrates the steps (send → in_flight → delivery when > d → redistribution every K messages) and shows the algorithm’s efficiency (low wasted %).

Readme (Testing)
From the project root:

```
python LessigRane-3-testing.py
```

- Runs scenarios **S.1** (one random sender), **S.2** (two random senders), **S.m** (all m parties).
- For each scenario: average wasted pads and average rounds over several trials.
- Execution ends when at least one active party can’t send securely (single pad sequence per run).

Optional arguments:

Argument	Meaning	Default

`-n, --pads`	Pad sequence length n	1000
`-d`	Max undelivered messages	From protocol
`-m`	Number of parties	From protocol
`--trials`	Trials per scenario	100
`--seed`	Random seed for reproducibility	42

Example:

```
python LessigRane-3-testing.py -n 500 -d 5 --trials 200 --seed 123
```

Results

```
--- m=3, n=1000, d=5, L=1, trials=5 ---
  S.1 (x=1): avg wasted = 26.6 (2.66%), avg rounds = 973.4, time = 0.02s
    async efficiency: avg redistributions = 48.0, avg messages per redistribution = 20.3
    chat sim: avg deliveries (receive calls) = 1936.8
  S.2 (x=2): avg wasted = 11.0 (1.10%), avg rounds = 989.0, time = 0.02s
    async efficiency: avg redistributions = 48.8, avg messages per redistribution = 20.3
    chat sim: avg deliveries (receive calls) = 1968.0
  S.3 (x=3): avg wasted = 4.0 (0.40%), avg rounds = 996.0, time = 0.02s
    async efficiency: avg redistributions = 49.0, avg messages per redistribution = 20.3
    chat sim: avg deliveries (receive calls) = 1982.0
  (m=3 total: 0.07s)

--- m=4, n=1000, d=5, L=1, trials=5 ---
  S.1 (x=1): avg wasted = 45.0 (4.50%), avg rounds = 955.0, time = 0.02s
    async efficiency: avg redistributions = 47.0, avg messages per redistribution = 20.3
    chat sim: avg deliveries (receive calls) = 2850.0
  S.2 (x=2): avg wasted = 21.4 (2.14%), avg rounds = 978.6, time = 0.02s
    async efficiency: avg redistributions = 48.2, avg messages per redistribution = 20.3
    chat sim: avg deliveries (receive calls) = 2920.8
  S.4 (x=4): avg wasted = 8.8 (0.88%), avg rounds = 991.2, time = 0.03s
    async efficiency: avg redistributions = 48.8, avg messages per redistribution = 20.3
    chat sim: avg deliveries (receive calls) = 2958.6
  (m=4 total: 0.07s)

--- m=5, n=2500, d=25, L=1, trials=5 ---
  S.1 (x=1): avg wasted = 64.0 (2.56%), avg rounds = 2436.0, time = 0.12s
    async efficiency: avg redistributions = 121.0, avg messages per redistribution = 20.1
    chat sim: avg deliveries (receive calls) = 9680.0
  S.2 (x=2): avg wasted = 25.6 (1.02%), avg rounds = 2474.4, time = 0.12s
    async efficiency: avg redistributions = 123.0, avg messages per redistribution = 20.1
```

```
    chat sim: avg deliveries (receive calls) = 9840.0
  S.5 (x=5): avg wasted = 6.0 (0.24%), avg rounds = 2494.0, time = 0.12s
    async efficiency: avg redistributions = 124.0, avg messages per redistribution = 20.1
    chat sim: avg deliveries (receive calls) = 9920.0
  (m=5 total: 0.37s)

--- m=10, n=5000, d=50, L=1, trials=5 ---
  S.1 (x=1): avg wasted = 162.0 (3.24%), avg rounds = 4838.0, time = 0.44s
    async efficiency: avg redistributions = 241.0, avg messages per redistribution = 20.1
    chat sim: avg deliveries (receive calls) = 43380.0
  S.2 (x=2): avg wasted = 87.0 (1.74%), avg rounds = 4913.0, time = 0.45s
    async efficiency: avg redistributions = 244.8, avg messages per redistribution = 20.1
    chat sim: avg deliveries (receive calls) = 44064.0
  S.10 (x=10): avg wasted = 22.0 (0.44%), avg rounds = 4978.0, time = 0.45s
    async efficiency: avg redistributions = 248.2, avg messages per redistribution = 20.1
    chat sim: avg deliveries (receive calls) = 44676.0
  (m=10 total: 1.34s)

--- m=100, n=50000, d=100, L=1, trials=5 ---
  S.1 (x=1): avg wasted = 1909.0 (3.82%), avg rounds = 48091.0, time = 41.43s
    async efficiency: avg redistributions = 2403.6, avg messages per redistribution = 20.0
    chat sim: avg deliveries (receive calls) = 4759128.0
  S.2 (x=2): avg wasted = 1205.8 (2.41%), avg rounds = 48794.2, time = 42.04s
    async efficiency: avg redistributions = 2438.8, avg messages per redistribution = 20.0
    chat sim: avg deliveries (receive calls) = 4828824.0
  S.100 (x=100): avg wasted = 176.4 (0.35%), avg rounds = 49823.6, time = 45.90s
    async efficiency: avg redistributions = 2490.6, avg messages per redistribution = 20.0
    chat sim: avg deliveries (receive calls) = 4931388.0
  (m=100 total: 129.36s)
```

For m = 3 in our testing, the protocol shows strong efficiency. 1 random sender (S.1) averages 26.6 wasted pads (2.66% of n) and 973 rounds per run—so a single active sender still uses almost 94% of the pad sequence before the run stops. Two random senders (S.2) do better: 11 wasted (1.10%) and 989 rounds, and all three parties active (S.3) is best: 4.0 wasted (0.40%) and 996 rounds. In all the cases, the percentage of pads wasted is low and decreases with the increase in the number of active parties, from 2.66% in the case of S.1 to 1% in the case of S.3, while the number of rounds remains high (930-990), meaning the pads are nearly fully utilized in the sequence. Redistributions remain around 48 per run, with an average of 20 messages redistributed, meaning the sync points occur rarely and the protocol is mostly asynchronous. In all the cases, the number of pads wasted is much lower than the naive upper bound of (m-1)*n/m pads, or 66.7%, and stays below the maximum of 7 (d+m-1) when all three are participating.

Extra credit

ec1: Generalization to the other value of m in {3, 4}

We implemented the same protocol for both m = 3 and m = 4. No change to the algorithm is required: m is a parameter everywhere (initial split, redistribution split, and who can send). For m = 4 we use the same scenarios as in the assignment: S.1 (one random sender), S.2 (two random senders), and S.4 (all four parties). Testing is run with the same n, d, L and the same definition of a run (single pad sequence, random senders, stop when at least one active party cannot send securely). The main claims hold for m = 4 as well: wasted pads are at most d + m − 1 in all schedules, and in our S.1/S.2/S.4 experiments the average wasted pads remain a small fraction of n (e.g. on the order of a few to low tens of percent for S.1, lower for S.2 and S.4). The readme and testing instructions apply to both m = 3 and m = 4; the chosen m can be set in the protocol (e.g. M = 4) or via the testing script (e.g. python testing.py -m 4 or --sweep-m).

ec2: Generalization to any m smaller than n/d

The protocol is written for arbitrary m: all data structures and steps use m as a parameter (number of parties, size of segments, number of chunks in redistribution). So the design already applies to any m ≥ 2.
The condition m < n/d is a natural feasibility constraint:
There are n pads and at most d pads can be “in flight” at once, so at least n − d pads are available for assignment.

- We need to assign pads to m parties. If m ≥ n/d then m · d ≥ n, so in the worst case (e.g. each party holding about d pads and all in flight) we could need n pads just for in-flight use, leaving nothing for future assignment. Requiring m < n/d ensures m · d < n, so there is always a positive number of pads left to redistribute after draining in-flight messages, and the protocol can make progress.
- Under m < n/d, the same invariants hold: no pad is ever reused (perfect secrecy), and the same proof shows that wasted pads are at most d + m − 1. Sending one message remains O(L) per party. Testing can be run for any such m (e.g. m = 5, 10, 100) with n and d chosen so that m < n/d (e.g. n = 500m, d = max(5, min(100, n/100)) as in our sweep). We have run the protocol and tests for several m in this range and observe that average wasted pads stay a small fraction of n across scenarios S.1, S.2, and S.m.

Use of Tools
Used AI tools for help with simulation and testing.